

EcoShop Capstone - Testing Guide

Testing Overview

This guide covers all aspects of testing the EcoShop application including functional testing, performance testing, security testing, and deployment verification.

1. Functional Testing

1.1 Authentication & User Management

Login Flow

- [] **Test:** Login with valid credentials
 - **Steps:**
 1. Open application
 2. Enter username (any text, e.g., "john")
 3. Enter password (any text, e.g., "demo123")
 4. Click Login

- **Expected:** Modal closes, user greeting displays, header updates
 - **Result:**  PASS /  FAIL
- [] **Test:** Login button disabled state
 - **Steps:**
 1. Open application
 2. Try clicking login with empty fields
 - **Expected:** Form doesn't submit, HTML5 validation shows
 - **Result:**  PASS /  FAIL
- [] **Test:** Session persistence
 - **Steps:**
 1. Login with credentials
 2. Close browser/tab
 3. Reopen application
 - **Expected:** User still logged in, greeting displays
 - **Result:**  PASS /  FAIL

Logout Flow

- [] **Test:** Logout removes session
 - **Steps:**
 1. Click Logout button

2. Confirm in dialog

- **Expected:** Login modal appears, user data cleared

◦ **Result:**  PASS /  FAIL

- [] **Test:** Logout cancellation

- **Steps:**

1. Click Logout button
2. Click Cancel in dialog

- **Expected:** User remains logged in

◦ **Result:**  PASS /  FAIL

1.2 Product Management (CRUD)

Create (Add Product)

- [] **Test:** Add product with all fields

- **Steps:**

1. Click "+ Add Product" button
2. Fill in all fields:
 - Name: "Eco Water Bottle"
 - Description: "Sustainable bamboo bottle"
 - Price: "29.99"
 - Stock: "50"

- Emoji: "🌿"

3. Click "Save Product"

- **Expected:**

- Modal closes
- Product appears in grid
- Success alert shows

- **Result:**  PASS /  FAIL

- [] **Test:** Add product with missing fields

- **Steps:**

1. Click "+ Add Product"
2. Leave Name or Description empty
3. Try to submit

- **Expected:** Form doesn't submit, validation message

- **Result:**  PASS /  FAIL

- [] **Test:** Add product with invalid price

- **Steps:**

1. Enter negative price
2. Try to submit

- **Expected:** Form rejects negative value

- **Result:**  PASS /  FAIL

- [] **Test:** Add product updates product count

- **Steps:**
 1. Note current product count
 2. Add new product
 3. Check grid for new product
- **Expected:** New product appears at end of grid
- **Result:**  PASS /  FAIL

Read (View Products)

- [] **Test:** Products display in grid
 - **Steps:**
 1. Open application
 2. Observe products view
 - **Expected:**
 - All products visible
 - Correct layout (responsive grid)
 - All product details show
 - **Result:**  PASS /  FAIL
- [] **Test:** Stock status displays
 - **Steps:**
 1. View different products with different stock levels
 - **Expected:**

- Stock > 20: "In Stock (X)"
- Stock 1-20: "Low Stock (X)"
- Stock 0: "Out of Stock"

◦ **Result:**  PASS /  FAIL

• [] **Test:** Product emoji displays

◦ **Steps:**

1. View products

◦ **Expected:** Each product shows its emoji in image area

◦ **Result:**  PASS /  FAIL

Update (Edit Product)

• [] **Test:** Edit product details

◦ **Steps:**

1. Click "Edit" on any product
2. Change Name to "Updated Name"
3. Change Price to "19.99"
4. Click "Save Product"

◦ **Expected:**

- Modal closes
- Grid updates with new details
- Success alert shows

- **Result:**  PASS /  FAIL
- [] **Test:** Edit product stock
 - **Steps:**
 1. Edit product
 2. Change Stock to "10"
 3. Save
 - **Expected:** Stock displays as "Low Stock (10)"
 - **Result:**  PASS /  FAIL
- [] **Test:** Cancel edit
 - **Steps:**
 1. Click "Edit"
 2. Change fields
 3. Click "Cancel"
 - **Expected:** Modal closes, changes discarded
 - **Result:**  PASS /  FAIL

Delete (Remove Product)

- [] **Test:** Delete product
 - **Steps:**
 1. Click "Delete" on any product
 2. Confirm in dialog
 - **Expected:**

- Product removed from grid
 - Success alert shows
 - **Result:**  PASS /  FAIL
- [] **Test:** Cancel deletion
 - **Steps:**
 1. Click "Delete"
 2. Click Cancel in dialog
 - **Expected:** Product remains in grid
 - **Result:**  PASS /  FAIL
- [] **Test:** Delete updates product count
 - **Steps:**
 1. Count products
 2. Delete one
 3. Recount
 - **Expected:** Count decreased by 1
 - **Result:**  PASS /  FAIL

1.3 Shopping Cart

Add to Cart

- [] **Test:** Add single item
 - **Steps:**

1. Click "Add to Cart" on any product

- **Expected:**

- Cart badge shows "1"
- Success alert displays

- **Result:** PASS / FAIL

- [] **Test:** Add same item twice

- **Steps:**

1. Click "Add to Cart" on Product A
2. Click "Add to Cart" on same Product A again

- **Expected:**

- Cart badge shows "1" (not 2)
- Quantity increased when viewing cart

- **Result:** PASS / FAIL

- [] **Test:** Add to cart with insufficient stock

- **Steps:**

1. Add product to cart multiple times
2. Try adding more than stock allows

- **Expected:** Error alert shows "Insufficient stock"

- **Result:** PASS / FAIL

View Cart

- **[] Test:** Cart displays items

- **Steps:**

1. Add product to cart
2. Click "Cart" button

- **Expected:**

- View switches to cart
- Item displays with:
 - Product emoji
 - Product name
 - Unit price
 - Quantity
 - Subtotal

- **Result:** PASS / FAIL

- **[] Test:** Cart summary calculates

- **Steps:**

1. Add items to cart
2. View cart

- **Expected:**

- Subtotal = sum of all (price × quantity)
- Tax = Subtotal × 0.10
- Total = Subtotal + Tax

- **Result:** PASS / FAIL

- [] **Test:** Empty cart state
 - **Steps:**
 1. Don't add anything to cart
 2. Click "Cart" button
 - **Expected:** Empty state displays with message and "Continue Shopping" button
 - **Result:**  PASS /  FAIL

Update Quantity

- [] **Test:** Increase quantity with + button
 - **Steps:**
 1. Add item to cart
 2. View cart
 3. Click + button next to quantity
 - **Expected:**
 - Quantity increases by 1
 - Totals recalculate
 - **Result:**  PASS /  FAIL
- [] **Test:** Decrease quantity with - button
 - **Steps:**
 1. Add item to cart (quantity 2+)
 2. View cart

3. Click - button

- **Expected:**

- Quantity decreases by 1
- Totals recalculate

- **Result:**  PASS /  FAIL

- [] **Test:** Set quantity via input

- **Steps:**

1. Add item to cart
2. View cart
3. Click in quantity field
4. Change to different number
5. Press Enter or click elsewhere

- **Expected:** Quantity updates, totals recalculate

- **Result:**  PASS /  FAIL

- [] **Test:** Quantity can't go below 1

- **Steps:**

1. Add item to cart
2. Try setting quantity to 0

- **Expected:** Item removed from cart instead

- **Result:**  PASS /  FAIL

Remove from Cart

- [] **Test:** Remove single item
 - **Steps:**
 1. Add multiple items to cart
 2. View cart
 3. Click "Remove" on one item
 - **Expected:**
 - Item removed from cart
 - Cart badge updates
 - Totals recalculate
 - **Result:**  PASS /  FAIL
- [] **Test:** Cart persists after refresh
 - **Steps:**
 1. Add items to cart
 2. Refresh page
 - **Expected:** Items still in cart
 - **Result:**  PASS /  FAIL

1.4 Orders & Checkout

Checkout

- [] **Test:** Successful checkout
 - **Steps:**
 1. Add items to cart

2. View cart
3. Click "Checkout" button

- **Expected:**
 - Order created
 - Order number displayed in alert
 - Cart cleared
 - Redirected to Orders view
 - Order appears in list
- **Result:**  PASS /  FAIL

- **[] Test:** Checkout with empty cart

- **Steps:**
 1. View empty cart
 2. Try clicking "Checkout"
- **Expected:** Error alert "Cart is empty"
- **Result:**  PASS /  FAIL

- **[] Test:** Stock deducted after order

- **Steps:**
 1. Note product stock before
 2. Add product to cart, checkout
 3. Back to products view
 4. Check product stock after
- **Expected:** Stock reduced by quantity ordered

- **Result:**  PASS /  FAIL

View Orders

- [] **Test:** Order appears in history
 - **Steps:**
 1. Create an order
 2. Click "Orders" button
 - **Expected:**
 - Order displays in list
 - Shows Order #ID
 - Shows date
 - Shows item count
 - Shows status (confirmed)
 - Shows total price
 - **Result:**  PASS /  FAIL
- [] **Test:** View order details
 - **Steps:**
 1. Click "View" on any order
 - **Expected:** Modal/alert shows:
 - Order number
 - Status
 - Date and time
 - List of items

- Quantities and prices
- Subtotal, tax, total
- **Result:**  PASS /  FAIL

Cancel Order

- [] **Test:** Cancel confirmed order
 - **Steps:**
 1. Create an order
 2. View Orders
 3. Click "Cancel" on order
 4. Confirm
 - **Expected:**
 - Order status changes to "cancelled"
 - Stock restored to products
 - Success alert shows
 - **Result:**  PASS /  FAIL
- [] **Test:** Cannot cancel already cancelled order
 - **Steps:**
 1. Cancel an order
 2. Try to cancel again
 - **Expected:** "Cancel" button not visible or disabled

- **Result:**  PASS /  FAIL
 - [] **Test:** Stock restored correctly
 - **Steps:**
 1. Note initial stock
 2. Order product
 3. Note stock after order
 4. Cancel order
 5. Check stock again
 - **Expected:** Stock returns to original amount
 - **Result:**  PASS /  FAIL
-

2. Responsive Design Testing

Mobile (320px - iPhone 5)

- [] **Layout:** Single column product grid
 - **Expected:** Products stack vertically
 - **Result:**  PASS /  FAIL
- [] **Navigation:** Header elements stack or collapse
 - **Expected:** No horizontal scrolling
 - **Result:**  PASS /  FAIL
- [] **Forms:** Inputs take full width

- **Expected:** Easy to tap on mobile
- **Result:**  PASS /  FAIL
- [] **Cart:** Touch-friendly quantity buttons
 - **Expected:** Buttons are large enough to tap
 - **Result:**  PASS /  FAIL

Tablet (768px - iPad)

- [] **Layout:** Two column product grid
 - **Expected:** 2 products per row
 - **Result:**  PASS /  FAIL
- [] **Navigation:** Horizontal layout
 - **Expected:** All nav items visible
 - **Result:**  PASS /  FAIL

Desktop (1024px - Large Screen)

- [] **Layout:** Three to four column grid
 - **Expected:** Multiple products per row
 - **Result:**  PASS /  FAIL
- [] **Cart Summary:** Beside cart items
 - **Expected:** Summary visible simultaneously
 - **Result:**  PASS /  FAIL

Extra Wide (1440px+)

- ☐ **Layout:** Four column grid
 - **Expected:** Balanced layout
 - **Result:** ☒ PASS / ☒ FAIL
- ☐ **Max width:** Container doesn't stretch too wide
 - **Expected:** Readable, professional appearance
 - **Result:** ☒ PASS / ☒ FAIL

Responsive Testing Checklist

- ☐ No horizontal scrollbars at any breakpoint
 - ☐ Text readable at all sizes
 - ☐ Images/emojis scale appropriately
 - ☐ Buttons touchable on mobile
 - ☐ No overlapping elements
 - ☐ Proper spacing maintained
-

3. Cross-Browser Testing

Test on all major browsers:

Chrome/Chromium

- ☐ Latest version
 - **Test:** Run all functional tests
 - **Result:** ☒ PASS / ☒ FAIL

Firefox

- ☐ Latest version
 - **Test:** Run all functional tests
 - **Result:** ☒ PASS / ☒ FAIL

Safari

- ☐ Latest version (macOS)
- ☐ Safari Mobile (iOS)
 - **Test:** Run all functional tests
 - **Result:** ☒ PASS / ☒ FAIL

Edge

- ☐ Latest Chromium-based version
 - **Test:** Run all functional tests
 - **Result:** ☒ PASS / ☒ FAIL

Browser-Specific Issues

- ☐ localStorage works in all browsers
 - ☐ CSS Grid/Flexbox renders correctly
 - ☐ Animations smooth (no janky)
 - ☐ Form validation works
 - ☐ Modals display properly
-

4. Performance Testing

Page Load Performance

- [] **First Contentful Paint (FCP):** < 1 second
 - **Tool:** Chrome DevTools → Lighthouse
 - **Result:**  PASS /  FAIL
- [] **Largest Contentful Paint (LCP):** < 2.5 seconds
 - **Tool:** Chrome DevTools → Lighthouse
 - **Result:**  PASS /  FAIL
- [] **Cumulative Layout Shift (CLS):** < 0.1
 - **Tool:** Chrome DevTools → Lighthouse
 - **Result:**  PASS /  FAIL

Lighthouse Audit

```
# Run Chrome DevTools Lighthouse
1. Open Chrome DevTools (F12)
2. Click Lighthouse tab
3. Click "Generate report"
4. Verify scores:
```

- [] **Performance:** ≥ 90
- [] **Accessibility:** ≥ 90
- [] **Best Practices:** ≥ 90
- [] **SEO:** ≥ 90

Bundle Size

- ☐ **HTML file:** < 100KB
 - ☐ **CSS:** Inline (no external files)
 - ☐ **JavaScript:** < 50KB
 - ☐ **Total:** < 150KB
-

5. Accessibility Testing

Keyboard Navigation

- ☐ **Tab through elements:** All interactive elements accessible
- ☐ **Enter/Space:** Buttons activate with Enter
- ☐ **Escape:** Modals close with Escape key
- ☐ **Focus visible:** Clear focus indicator on all elements

Screen Reader Testing (optional)

- ☐ Headings properly marked (h1, h2, etc.)
- ☐ Images/emojis have alt text context
- ☐ Form labels associated with inputs
- ☐ Buttons have descriptive text

Color Contrast

- ☐ Text contrast $\geq 4.5:1$ for normal text
- ☐ Text contrast $\geq 3:1$ for large text

- ☐ No color-only information conveyed

Form Accessibility

- ☐ All inputs have labels
 - ☐ Required fields marked
 - ☐ Error messages clear and associated
-

6. Security Testing

Input Validation

- ☐ **XSS Prevention:**
 - Try entering `<script>alert('xss')</script>` in product name
 - **Expected:** Displayed as text, not executed
 - **Result:** ☒ PASS / ☐ FAIL
- ☐ **SQL Injection** (N/A - no backend)
 - **Note:** Not applicable, all data local

localStorage Security

- ☐ **No sensitive data stored:**
 - Open DevTools → Application → Storage
 - **Check:** No passwords, no tokens stored
 - **Result:** ☒ PASS / ☐ FAIL

- [] **HTTPS enforcement** (production):
 - **Expected:** All requests over HTTPS
 - **Result:**  PASS /  FAIL

CSRF Protection (N/A)

- **Note:** Not applicable, no cross-site requests
-

7. Data Persistence Testing

localStorage Operations

- [] **User data persists:**
 1. Login as user
 2. Refresh page
 3. User still logged in
 - **Result:**  PASS /  FAIL
- [] **Products persist:**
 1. Add product
 2. Refresh page
 3. Product still there
 - **Result:**  PASS /  FAIL
- [] **Cart persists:**

1. Add to cart
2. Refresh page
3. Items still in cart

◦ **Result:**  PASS /  FAIL

- **[] Orders persist:**

1. Create order
2. Refresh page
3. Order still in history

◦ **Result:**  PASS /  FAIL

Storage Quota

- **[] Handles full storage gracefully:**

1. Add many large items
2. Should error gracefully, not crash

◦ **Result:**  PASS /  FAIL

8. Browser DevTools Testing

Console Check

```
# Open DevTools (F12)
# Check Console tab
```

- ☐ No JavaScript errors (red messages)
- ☐ No warning messages (yellow messages)
- ☐ No CORS errors
- ☐ No 404 errors for resources

Network Tab

- ☐ All files load successfully (200 status)
- ☐ No failed requests
- ☐ Files load within reasonable time
- ☐ No unnecessary requests

Application Tab

- ☐ localStorage keys visible
- ☐ Data properly formatted (JSON)
- ☐ No corrupted data

9. Edge Cases & Error Handling

- ☐ **Out of stock:**
 - Add product with 0 stock to cart
 - **Expected:** Error shown
 - **Result:** ☒ PASS / ☐ FAIL
- ☐ **Insufficient stock:**

- Try to order more than available
 - **Expected:** Error shown
 - **Result:**  PASS /  FAIL
 - **[] Missing data:**
 - Try submitting form with empty fields
 - **Expected:** Validation error
 - **Result:**  PASS /  FAIL
 - **[] Rapid clicks:**
 - Click buttons very quickly
 - **Expected:** No duplicate actions
 - **Result:**  PASS /  FAIL
 - **[] Multiple tabs:**
 - Open app in multiple tabs
 - **Expected:** Data consistent across tabs
 - **Result:**  PASS /  FAIL
-

10. Production Deployment Testing

Pre-Deployment

- **[]** All code committed to Git
- **[]** No sensitive data in repo
- **[]** Documentation complete

- ☐ ☐ Lighthouse score ≥ 90

Deployment

- ☐ ☐ Successfully deployed to platform
- ☐ ☐ Live URL accessible
- ☐ ☐ No 404 errors

Post-Deployment

- ☐ **Test live URL in browser**
 - Run full functional test suite (section 1)
 - **Result:** ☒ PASS / ☒ FAIL
- ☐ **Test on mobile device**
 - Use actual phone if possible
 - **Result:** ☒ PASS / ☒ FAIL
- ☐ **Performance on live**
 - Run Lighthouse on live URL
 - **Expected:** Scores ≥ 90
 - **Result:** ☒ PASS / ☒ FAIL

Test Report Template

EcoShop Capstone - Test Report

```
**Date**: [Date]
**Tester**: [Name]
**Environment**: [Browser/Device]
**Build Version**: [Version]

## Summary
- Total Tests: X
- Passed: X
- Failed: X
- Pass Rate: X%

## Functional Testing
- Authentication:  PASS /  FAIL
- Products (CRUD):  PASS /  FAIL
- Shopping Cart:  PASS /  FAIL
- Orders:  PASS /  FAIL

## Responsive Design
- Mobile (320px):  PASS /  FAIL
- Tablet (768px):  PASS /  FAIL
- Desktop (1024px):  PASS /  FAIL

## Performance
- Lighthouse Score: [Score]
- Bundle Size: [Size]
- Load Time: [Time]

## Issues Found
[List any issues]

## Recommendations
[Any suggestions for improvement]
```

Sign-off

Tester: _____ Date: _____

Continuous Testing Checklist

Every deployment should include:

- ☐ Functional tests (Section 1)
- ☐ Responsive tests (Section 2)
- ☐ Performance tests (Section 4)
- ☐ Security tests (Section 6)
- ☐ Cross-browser test (at least Chrome, Firefox, Safari)
- ☐ localStorage tests (Section 7)
- ☐ Console check (no errors)
- ☐ Lighthouse report (score $\geq$ 90)

Testing Guide Version: 1.0

Last Updated: September 24, 2026